

Piles multiples d'exécution

MU4IN501 – DLP : Développement d'un langage de programmation
Master STL, Sorbonne Université

Antoine Miné

Cours 7bis (préparation au TME 7)

24 octobre 2023

Année 2023–2024

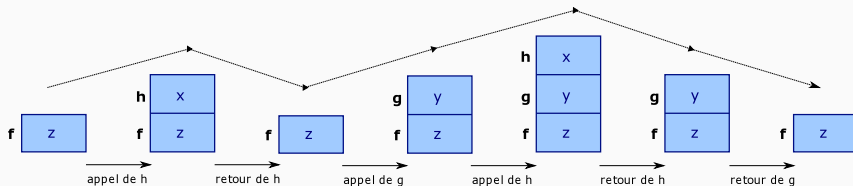


Rappel : modèle d'exécution par pile (en C)

```
void h(int x) {  
}  
void g(int y) {  
    h(y+1);  
}  
void f() {  
    int z;  
    h(2);  
    g(3);  
}
```

L'exécution des appels de fonctions forme une **pile** de **blocs d'activation**.

- **appel** : **empilage** des arguments, variables locales, temporaires, adresse de retour ;
- **retour** : **dépilage**,
saut à l'adresse de retour.



Rappel : retour de fonction

retour anticipé

```
void f() {  
    while(...) {  
        if (...) return;  
        ...  
    }  
}
```

saut long

```
jmp_buf buf;  
void f() {  
    if (setjmp(buf)==0) {  
        g();  
    }  
}  
void g() {  
    h();  
    ...  
}  
void h() {  
    if (...)  
        longjmp(buf);  
}
```

Il est facile de :

- sortir en tout point d'une fonction vers l'appelant direct ;
pas uniquement en fin de fonction
- dépiler plusieurs blocs d'activation en une fois, faire un saut calculé
avec `setjmp` et `longjmp`, voir le cours sur les exceptions

Limitation du modèle à pile unique

appel de coroutine

```
void f() {  
    c = call g(12);  
    ...  
    resume c;  
    ...  
    resume c;  
    ...  
}
```

coroutine

```
void g(int n) {  
    for (int i=0; i<n; i++) {  
        yield();  
    }  
}
```

Par contre, les **coroutines** sont impossibles, i.e. :

- créer un appel à **g** en fixant la valeur des arguments (**call**)
- puis exécuter un morceau de **g** et retourner avant la fin (**yield**)
- puis exécuter la suite de **f** , après l'appel à **g**
- puis **reprendre** l'exécution de **g** là où elle s'était arrêtée,
en **retrouvant** les valeurs des **variables locales** (**restore**).

Raison : une fois le bloc d'activation dépilé, toutes ses variables locales disparaissent et ne peuvent pas être restaurées.

But : pouvoir interrompre une exécution et la reprendre.

Applications :

- gestion des erreurs avec réessai ;
(`try ...error ...catch { ...retry; }`)
- itérateurs de collection (e.g., les générateurs en Python) ;
- modèle producteur / consommateur (e.g., compression a volée) ;
- programmation réactive synchrone (e.g., systèmes embarqués, jeux vidéo) ;
- programmation concurrente.

Solutions possibles : c.f. TME 7

- créer explicitement plusieurs piles (en C) ;
- ou utiliser les processus légers : les *threads* (en Java ou en C).
- aussi : dans les langages fonctionnels, utilisation des continuations

Piles multiples en C

`ucontext` : interface de bas niveau en C de manipulation des piles.

- comme `longjmp`, permet de modifier le pointeur de pile ;
- permet également de **créer des piles séparées**, et de sauter d'une pile à l'autre.

Chaque pile comporte ses propres blocs d'activation et permet une séquence d'appels et de retours indépendante des autres piles.

C'est une **unité d'exécution de programme indépendante**.

- à la différence des *threads*, le changement de pile est contrôlé par le programme, pas par le système, et il n'y a pas d'exécution réellement parallèle multi-cœurs.

Interface `ucontext` (1/2)

L'interface est accessible par : `#include <ucontext.h>`

Elle contient un type :

- `ucontext_t`

information de contexte (similaire à `jmpbuf_t`)

contient un pointeur de pile et une copie des registres

et deux fonctions permettant de simuler `setjmp` et `longjmp` :

- `int getcontext (ucontext_t *ucp)`

initialise `ucp` avec l'information de contexte courant

similaire à `setjmp`

sauter à `ucp` revient à sauter à l'instruction suivant le retour de `getcontext`

- `int setcontext (ucontext_t *ucp)`

saute vers le contexte `ucp`

la fonction ne retourne jamais, comme `longjmp`

Exemple : simulation de saut long

```
#include <ucontext.h>

ucontext_t uc;

void f() {
    if (getcontext (&uc) < 0) // erreur ...
        g();
}

void g() {
    h();
}

void h() {
    setcontext(&uc);
}
```

Très similaire à `setjmp` et `longjmp`... pour l'instant.

Interface `ucontext` (2/2)

Les champs de `ucontext_t` permettent de gérer des **stacks multiples** **allouées par le programmeur** (pas par le système) :

- `uc_stack.ss_sp` : pointeur vers le début de la pile ;
- `uc_stack.ss_size` : taille de la pile ;
- `uc_link` : où sauter après un `return` quand la pile est vide.

Les fonctions suivantes sont alors utiles :

- `void makecontext (ucontext_t *ucp, void (*f) (void), int argc, ...)`

change la pile et le point de saut d'un contexte

la structure `ucp` doit être d'abord initialisée par `getcontext`,
puis les champs `uc_stack`, `uc_link` renseignés avant l'appel ;
sauter vers `ucp` revient alors à exécuter `f` dans la nouvelle pile

- `int swapcontext (ucontext_t* oucp, ucontext_t * ucp)`

stocke le contexte courant dans `oucp` et saute vers le contexte `ucp`

sauter vers `oucp` revient à sauter à l'instruction suivant le retour de `swapcontext`

Exemple : navigation entre piles multiples

```
#include <ucontext.h>

ucontext_t ucf, ucg;

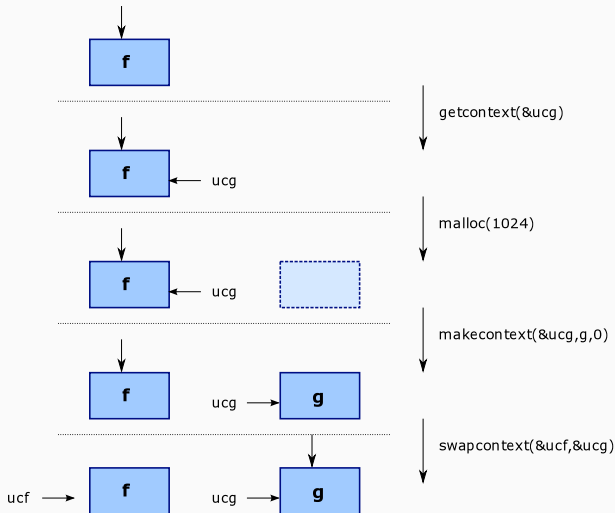
void f() {
    getcontext(&ucg);
    ucg.uc_stack.ss_sp = malloc(1024);
    ucg.uc_stack.ss_size = 1024;
    makecontext(&ucg, g, 0);
    // A ...
    swapcontext(&ucf, &ucg);
    // B ...
    swapcontext(&ucf, &ucg);
    // C ...
}

void g() {
    // D ...
    swapcontext(&ucg, &ucf);
    // E ...
    setcontext(&ucf);
    // F ...
}
```

Flot d'exécution : A, D, B, E, C.

- initialisation d'un contexte **ucg** ;
- une pile est allouée
et **ucg** est **attaché** à la pile ;
- **ucg** est **attaché** à la fonction **g** ;
- **échange** de contrôle
entre **ucg** et **ucf** ;
- puis entre **ucg** et **ucf**, etc. ;
- le contrôle **revient** à la fin à **ucf**,
donc à **f**.

Illustration de la création de piles multiples



Processus légers en Java

En système, un processus est une unité d'exécution indépendante, donc avec son propre flot de contrôle et sa propre pile.

- **processus** classique :
un programme, avec son propre espace mémoire isolé des autres ;
- processus « légers » ou **threads** :
unité d'exécution dans un programme,
créée dynamiquement,
partageant les variables globales et les ressources avec les autres threads.

Les processus s'exécutent en **concurrency** :

- parallélisme réel : sur des cœurs différents ;
- parallélisme simulé : par changements de contexte rapides ;
- ou un mélange des deux.

Un **ordonnanceur** contrôle quels processus s'exécutent à chaque instant.

(possibilité de vrai parallélisme sur multi-cœurs)

Plusieurs manières de faire en Java, plus ou moins haut niveau.

Manière utilisée ici : hériter de la classe `java.lang.Thread` (bas niveau) :

- la classe doit définir une méthode `void public run()` ;
point d'entrée de la thread
- une fois l'instance de thread créée, appeler sa méthode `start`.

Exemple de thread en Java

```
public class Activity extends Thread
{
    private int x;
    public Activity(int x) {
        this.x = x;
        start();
    }
    public void run() {
        // ...
    }
}

...
new Activity(1);
new Activity(2);
new Activity(3);
```


L'ordre d'exécution des threads est contrôlé par l'ordonnanceur ;
celui-ci choisit quelles threads non-bloquées s'exécutent effectivement.

⇒ ce n'est pas ce que l'on veut pour nos coroutines.

Le contrôle sur les threads se fait par des **primitives de synchronisation**
qui peuvent les **bloquer** !

Exemple : les sémaphores `java.util.concurrent.Semaphore`

- contient un compteur entier : nombre de ressources ;
- **acquire** : consomme une ressource (décrémente le compteur) ;
- **release** : produit une ressource (incrémente le compteur) ;
- le compteur ne **descend jamais en dessous de zéro** ;
acquire sur un compteur à zéro force la thread à attendre le prochain **release** !
⇒ contrôle de l'exécution des threads.

Interface très simple, mais très puissante. . .
de nombreuses utilisations en programmation concurrente.

Exemples d'utilisation de sémaphore

section critique

```
Semaphore x = new Semaphore(1);

void m1() {
    x.acquire();
    // section critique
    x.release();
}

void m2() {
    x.acquire();
    // section critique
    x.release();
}
```

signal

```
Semaphore x = new Semaphore(0);

void m1() {
    // initialisation
    x.release();
    // calcul, après initialisation
}

void m2() {
    x.acquire();
    // calcul, après initialisation
}
```

- **section critique** : une seule méthode parmi **m1** et **m2** s'exécute à un instant donné ;
- **signal** : **m1** signale à **m2** quand elle peut commencer son exécution.